

Hard-Level Python Operators Interview Questions

30 Interview Questions

1. Explain the difference between the `is` operator and the `==` operator when comparing two mutable objects.
2. How does Python's `@` matrix multiplication operator work with custom classes? Provide an example implementation of `__matmul__`.
3. Describe the short-circuit evaluation behavior of the `and` and `or` operators with side-effects. Give a code snippet that demonstrates this.
4. What is the precedence order between bitwise operators (`&`, `|`, `^`, `~`, `<<`, `>>`) and logical operators (`and`, `or`, `not`)? Provide an example where misunderstanding this order leads to a bug.
5. How can you overload the augmented assignment operators (e.g., `+=`, `*=`) in a user-defined class? Show the special methods involved.
6. Discuss the role of the `__richcmp__` methods (`__lt__`, `__le__`, `__gt__`, `__ge__`, `__eq__`, `__ne__`) in customizing comparison operators. Why might you need to implement all six?
7. Explain why the expression `a < b < c` is evaluated as a chained comparison and not as `(a < b) < c`. What are the performance implications?
8. In Python, the `in` operator can be used with many container types. How does it work under the hood for user-defined objects? Which special method must be implemented?
9. Demonstrate how the `:=` (walrus) operator can be used inside a list comprehension that filters and transforms data simultaneously.
10. Why does the expression `bool([]) + bool({})` return 0? Explain the underlying type coercion rules for arithmetic with booleans.
11. How does the `//` floor division operator behave with negative numbers? Provide examples that illustrate the difference from true division `/`.
12. Describe how the `*` operator works for both sequence repetition and argument unpacking. What rules does Python follow to disambiguate the two uses?
13. Explain the effect of using the `~` (bitwise NOT) operator on a Python integer. How does it relate to two's complement representation?
14. What is the result of `0 == False` and `0 is False`? Why does Python treat these cases differently?

15. How can you implement a custom class that supports the @ operator for both left-hand and right-hand matrix multiplication? Which dunder methods are required?
16. Discuss the potential pitfalls of using mutable default arguments with operators that modify the argument in place (e.g., list += other).
17. Explain how the pow() built-in function differs from the ** exponentiation operator when a third modulus argument is supplied.
18. Why does the expression 5 / 2 == 5 // 2 evaluate to False in Python 3 but not in Python 2?
19. Show how to overload the << and >> operators for a custom bit-vector class, and discuss any constraints imposed by Python's data model.
20. How does the or operator determine the value it returns (as opposed to just a boolean)? Provide an example where this matters.
21. Explain the difference between a & b and a and b when a and b are sets. What does each expression evaluate to?
22. What is the purpose of the __invert__ method, and how does it interact with the ~ operator for user-defined numeric types?
23. Discuss how operator precedence changes when parentheses are introduced in a complex expression involving arithmetic, bitwise, and logical operators.
24. How can you use the * operator to unpack a dictionary into a function call (Python 3.5+), and what operator is used for merging dictionaries?
25. Provide a scenario where using the += operator on a tuple raises an error, and explain why.
26. Explain the behavior of the == operator when comparing a NumPy array with a scalar. How does this differ from Python's built-in list comparison?
27. How does Python's operator module expose the functionality of built-in operators? Give an example of using operator.attrgetter with the @ operator.
28. What is the result of bool('False') and why? Discuss the distinction between truthiness and boolean value of strings.
29. Show how to implement a class that supports the + operator for both left-hand and right-hand addition with built-in types (e.g., int + MyNumber). Which methods must be defined?
30. Explain why the expression 1 < 2 < 3 > 2 is valid and what its final boolean result is. Detail the evaluation order and short-circuiting behavior.